MARL-GPT: Foundation Model for Multi-Agent Reinforcement Learning

Anonymous Author(s)
Submission Id: «submission id»

ABSTRACT

Recent advances in multi-agent reinforcement learning (MARL) have demonstrated success in numerous challenging domains and environments, but typically require specialized models for each task. In this work, we propose a coherent methodology that makes it possible for a single GPT-based model to learn and perform well across diverse MARL environments and tasks, including StarCraft Multi-Agent Challenge, Google Research Football and POGEMA. Our method, MARL-GPT, applies offline reinforcement learning to train at scale on the expert trajectories (400M for SMACv2, 100M for GRF, and 1B for POGEMA) combined with a single transformer-based observation encoder that requires no task-specific tuning. Experiments show that MARL-GPT achieves competitive performance compared to specialized baselines in all tested environments. Thus, our findings suggest that it is, indeed, possible to build a multi-task transformer-based model for a wide variety of (significantly different) multi-agent problems paving the way to the fundamental MARL model (akin to ChatGPT, Llama, Mistral etc. in natural language modeling).

Code and Appendix: https://anonymous.4open.science/r/marl-gpt-20365

KEYWORDS

Multi-agent Learning, Reinforcement Learning, Multi-Task Learning

ACM Reference Format:
Anonymous Author(s). 2026. MARL-GPT: Foundation Model for Multi-Agent Reinforcement Learning. In *Proc. of the 25th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2026), Paphos, Cyprus, May 25 – 29, 2026*, IFAAMAS, 13 pages.

1 INTRODUCTION

Multi-agent reinforcement learning (MARL) has made significant progress in solving complex tasks such as competitive games like StarCraft or cooperative robotic tasks such as multi-agent pathfinding. However most MARL methods are designed for a single environment or task, requiring specialized architectures and training pipelines for each new problem. And even in this case the resultant MARL policies typically struggle when solving the problems not seen in the training time that are different in the number of agents, map layout etc. In this work, we wish not only to improve the generalization abilities of a MARL policy but to make it possible for such policy to efficiently solve tasks from different domains.

To this end we leverage the power of transformer-based architectures (that are the backbone of the overwhelming success in natural language processing tasks and are successfully applied to other domains such as computer vision, robotic control as well) coupled with imitation-learning at scale. We focus on three significantly different MARL environments: Starcraft Multi-Agent Challenge (adversarial combat game) [28], Google Research Football (adversarial sport game) [15], and POGEMA (cooperative multi-robot navigation) [31]. Using the expert policies for those environments that include both learnable policies and the rule-based ones, we obtain a large and diverse dataset of observation-action pairs that are used to train our model, which we call MARL-GPT.

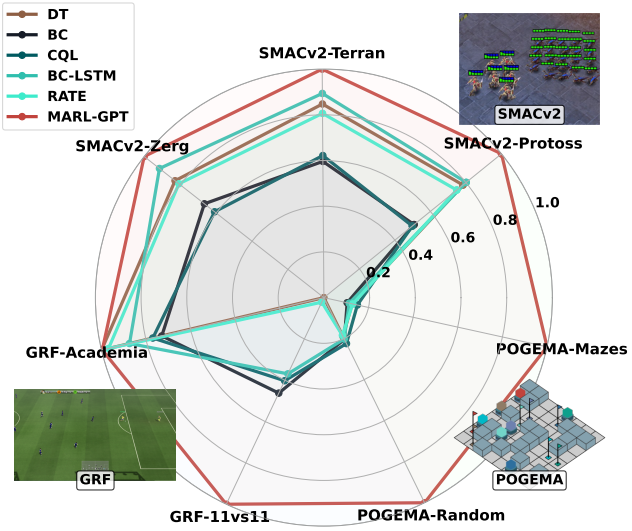

Figure 1: Spider plot demonstrating the relative performance of all evaluated approaches on three different environments – SMACv2, Google Research Football (GRF) and POGEMA.

We train MARL-GPT using supervised imitation learning on a diverse dataset of expert demonstrations collected from multiple MARL environments and tasks. Each expert trajectory consists of sequences of observation-action-reward triplets gathered across scenarios that vary in dynamics, state spaces, and agent interactions. To represent these observations, we tokenize structured, vectorized inputs describing each agent's local view – including itself, allies, and opponents. These tokens are enriched with the learned embeddings that encode the type of feature (e.g., position, health), the agent's identity and team affiliation, and the temporal step of the observation. This flexible encoding supports a variable number of agents and preserves role-specific and group-specific information while maintaining permutation invariance where applicable. The resulting tokens are processed by a transformer encoder, which maps them to expert action predictions and Q-values. Training is performed using standard cross-entropy loss on discrete action

Proc. of the 25th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2026), C. Amato, L. Dennis, V. Mascardi, J. Thangarajah (eds.), May 25 – 29, 2026, Paphos, Cyprus. © 2026 International Foundation for Autonomous Agents and Multiagent Systems (www.ifaamas.org). This work is licenced under the Creative Commons Attribution 4.0 International (CC-BY 4.0) licence.

outputs or Q-value bins, without relying on recurrence or autoregression. Instead, partial observability is handled by leveraging the transformer's context window and temporal encoding.

As a result we are able to obtain a single (fundamental) MARL model, that generalizes across state and action spaces, reward structures, and agent coordination requirements as confirmed by our extensive empirical evaluation.

Our experiments show that this unified model achieves competitive performance compared to specialized MARL algorithms in all tested domains. This suggests that general-purpose architectures can handle diverse multi-agent tasks without extensive tuning, reducing the need for task-specific designs.

To summarize, our key contributions are as follows:

(1) We develop a single GPT-based MARL model that performs notably well across different environments without any architectural changes or fine-tuning.
(2) We create a large dataset of observation-action-reward triplets, vital for imitation learning and offline RL of any MARL policy (not necessarily ours).
(3) We conduct a thorough empirical validation of our model on SMACv2, Google Research Football, and POGEMA, showing competitive results against specialized baselines.

We are committed to open-sourcing the model code, including expert policy weights, training datasets, and final weights of the MARL-GPT model.

2 RELATED WORK

Multi-agent imitation learning (MAIL). In the realm of multi-agent systems, imitation learning and learning from demonstration are widely employed [20, 33]. Imitation learning in multi-agent scenarios, also known as MAIL, is a problem in which agents learn to perform a task in a multi-agent system by observing and mimicking expert demonstrations without any knowledge of the reward function from the environment. This approach has gained particular traction in the context of controlling urban traffic and traffic lights at intersections [2, 12], controlling the power in wireless networks [42] due to the availability of a vast amount of data collected in real-world scenarios and the use of high-quality simulators, e.g. Sumo [21] in traffic applications.

Within the realm of MAIL, there are various methods to consider, including those that employ Bayesian approaches [40], generative adversarial techniques [18, 32], statistical tools for capturing interdependencies between agents [36], low-rank subspaces [29], latent models for coordinating agents [16], decision transformers [22, 38], and more. Demonstrations are frequently used for pre-training in games, such as learnable models for chess [27, 30], and in multi-agent pathfinding tasks, as exemplified by MAPF-GPT [1] and SCRIMP [37]. Despite the presence of models that are trained on pre-collected data, they are often specialized for a specific environment or even a single task within that environment. These models have limited generalizability even to conditions within the same environment, and their performance is significantly reduced when learning in a multitasking setting.

Foundation models for multi-agent systems. Foundation models are typically trained on large datasets, enabling zero-shot or few-shot learning [3, 41]. From the perspective of an autonomous agent, it is a model that can perform new tasks that are different from those it was trained on, either with additional demonstration of desired behavior or without any demonstration [9]. Another important feature of these foundation models is their fine-tuning ability for specific tasks, allowing them to improve performance quickly (see Gato tuning [25]). These models are commonly used in robotics to solve multimodal tasks where the goal is specified in text instructions [9, 13, 34]. However, in multi-agent scenarios, such models are less common, with examples including the Magnetic-One model for language and multimodal task-solving in WebArena [10], as well as the MAPF-GPT model for pathfinding [1].

Work on multitasking multi-agent models is also close to this topic, in which attempts are made to train a single policy for action in different scenarios and environments [19, 24]. However, these studies either focus on the training of generalized overused skills without good overall performance or they do not assume the properties of foundation models with the possibility of fine-tuning in additional demonstrations.

Some papers have explored the possibility of adapting single-agent foundation models to solve multi-agent problems without modifying the pre-training process [35, 39]. In our method, we focus on the task of training a foundation model for multiple multi-agent environments from scratch. It should be noted that no such single pre-trained model has been proposed yet. This is due to the complex nature of multi-agent policies in various tasks, such as StarCraft [28] and football [15], and the lack of large expert trajectory datasets necessary for effective foundation model training. In our MARL-GPT model, we overcome these difficulties and show that it is possible to train a multitasking, multi-agent foundation model on a reasonable set of expert data and with expert-level quality.

3 BACKGROUND

Multi-Agent Reinforcement Learning extends single-agent reinforcement learning (RL) to domains with multiple interacting decision makers. In partially observable environments, this interaction can be formalized by a decentralized partially observable Markov decision process (Dec-POMDP) [23]. This is a tuple $(N, \mathcal{S}, \mathcal{A}, T, \{O^i\}_{i\in N}, O, \{\mathcal{R}^i\}_{i\in N}, \gamma)$, where N is the agent set, $\mathcal{S}$ is the global state space, and all agents share a common action space $\mathcal{A}$. The joint action is $\mathbf{a} = (a^1, \ldots, a^n) \in \mathcal{A}^n$. The transition kernel $T(s' \mid s, \mathbf{a})$ specifies the probability of moving from state s to s' given the joint action. Each agent receives a private observation $o^i \in O^i$ drawn from the observation function $O(o^1, \ldots, o^n \mid s, \mathbf{a}, s')$. Rewards may differ across agents, with $\mathcal{R}^i(s, \mathbf{a}, s')$ denoting the signal for agent i, and $\gamma \in [0, 1)$ the discount factor.

At each time step t, agent i selects its action based on its own interaction history via a policy $\pi^i : (o_0^i, a_0^i, \ldots, o_t^i) \mapsto \Delta(\mathcal{A})$, where $\Delta(\cdot)$ denotes the distribution of actions provided by the policy. After all agents act, the environment transitions and each agent receives $r_t^i = \mathcal{R}^i(s_t, \mathbf{a}_t, s_{t+1})$. The learning objective in MARL is for each agent i to find an optimal policy $\pi^{*\,i}$ that maximizes its expected cumulative return, which is a discounted sum of step-wise rewards.

Each agent deploys its learned policy, which selects actions based only on its own local observation history. The policies themselves are conditioned only on local information, making execution fully

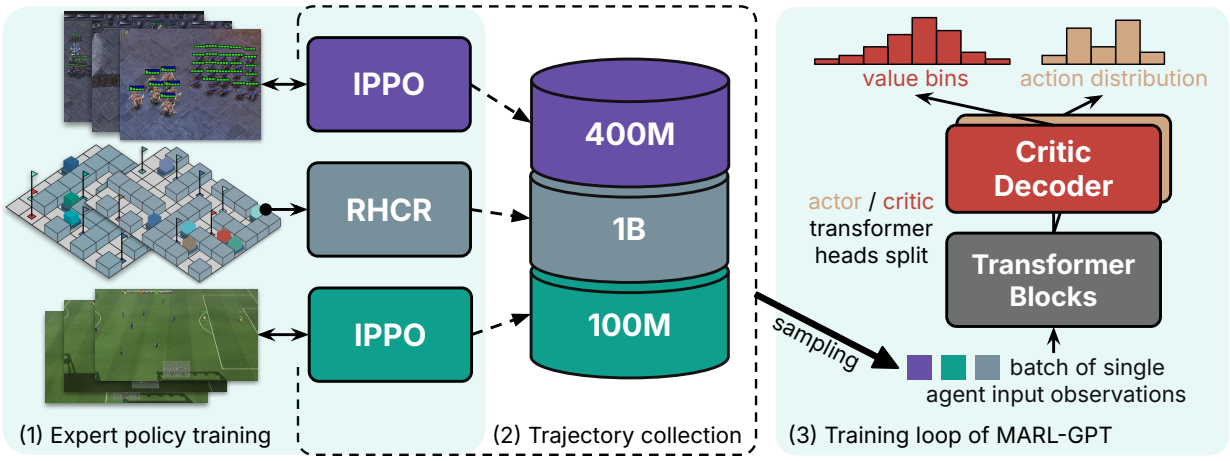

Figure 2: The general pipeline begins with training expert policies across diverse MARL environments using domain-appropriate algorithms (e.g., IPPO, RHCR). These policies generate large-scale datasets of observation-action(-reward) trajectories. MARL-GPT is then trained on the aggregated data using cross-entropy loss to predict expert actions and Q-values, enabling a single model to generalize across environments, tasks, and coordination regimes.

decentralized. Agents do not need to communicate or observe others' states/actions at runtime.

Parameter sharing employs a single policy network π across all agents. This approach reduces the parameter count and enables efficient knowledge transfer, though it may limit policy diversity when agents require specialized behaviors.

Offline RL and imitation learning methods train on expert trajectories $\mathcal{D}$. The simplest approach is behavior cloning, which formulates imitation learning as supervised learning by minimizing the cross-entropy loss $\mathbb{E}_{o_t,a_t\sim\mathcal{D}}\left[\mathcal{L}_{CE}(a \sim \pi(a|o_t), a_t)\right]$ over expert demonstrations (o_t, a_t).

4 METHOD

The suggested pipeline of obtaining a multi-task MARL model can be divided into three main components. First, expert policies for each task are acquired, and expert trajectories are collected using them. Second, observations are encoded in a way that enables the model to operate across multiple environments, in contrast to the expert policies, which are specialized for a single environment. Finally, a transformer architecture is trained via behavior cloning to imitate the expert behavior.

4.1 Acquiring of Expert Policies and Data Collection

Let π_e^* denote an expert policy for the environment $e \in \mathcal{E}$. These expert policies serve as sources of high-quality demonstrations and can be obtained through a variety of means. In certain well-established domains, such as chess, expert policies are readily available in the form of rule-based or search-based engines (e.g., Stockfish with NNUE), which do not require further training. These systems can generate large volumes of expert trajectories with negligible additional cost.

In contrast, expert policies for complex multi-agent systems are significantly less common. One exception is the domain of multi-agent pathfinding, where centralized solvers (e.g., Conflict-Based Search) can be used to compute globally optimal or near-optimal solutions. These solvers are environment-specific but provide valuable expert supervision when applicable.

In many modern MARL environments, no public expert policies are readily available. In such cases, we construct strong policies by applying large-scale reinforcement learning. For instance, we use independent PPO agents trained separately in each environment e to convergence, resulting in a set of high-performing task-specific policies. This approach is applicable even in partially observable settings.

Each expert policy π_e^* is used to collect trajectories by interacting with its corresponding environment. If the policy is recurrent, it may condition on a history of past observations. Specifically, the expert produces a trajectory:

$$\tau_e = \{(o_t, a_t, r_t)\}_{t=1}^{T},$$

where o_t is the observation at time t, and $a_t \sim \pi_e^*(\cdot \mid h_t)$ is the action sampled based on observation history $h_t = \{o_{t-k}, \ldots, o_t\}$. The agent receives the reward r_t. In practice, the policy may use a GRU or similar recurrent architecture to encode h_t into a compact hidden state.

We collect expert trajectories across a distribution of environments $\mathcal{E}$. Some of them may contain multiple different scenarios (tasks), allowing us to gather data that vary not only across environments, but also across tasks within a given environment. This results in a diverse dataset of expert demonstrations covering a wide range of state spaces, dynamics, and task structures.

4.2 Observation Encoding

We consider the task of multi-agent decision making in a partially observable environment. In this setting, observations usually include information about the agent itself, the surrounding agents, and the environment around them. We assume that observation from any environment can be represented as a vector $o = \{o_i\}_{i=0}^{n}$. For each element o_i in this observation vector, we obtain a token embedding tok_i by applying a learned linear layer $tok_i = L(o_i)$ that projects the raw input into the embedding space.

Each element of the observation corresponds to a specific environmental property, e.g. the health of the nearest agent. To provide context to these elements, we augment them with positional and structural information. Four positional numbers are defined to encode this information. First, pos_{attr} (attribute) indicates the type of observed property. For instance, whether it is the agent's health or its coordinates. If both agent k and agent j have the same property P observed, they will share the same pos_{attr}. Second, pos_{team} denotes the group to which the observed agent belongs. Agents may be divided into different groups based on their roles or goals, such as allies and enemies. Third, agents can be numbered within their group using pos_{indx} . Thus, an observation element o_i can be associated with an agent of the group pos_{team} and the index pos_{indx}. The agent whose observation is being considered is always assigned group 0 and index 0. All global properties of the situation are also attributed to this agent (group 0, index 0).

The environment is partially observable, which means that the agent does not have access to the full state of the environment. One simple way to handle this limitation is to include in the final observation a history of the last h observations from the environment, or to complete the current observation with some information from previous steps. To indicate the time moment of each observation element, we add a positional encoding pos_{time}. It is important to note that we do not use methods such as autoregressive modeling or recurrent neural networks in this approach.

Thus, each observation element o_i can be represented by a tuple of four components: agent index pos^i_{indx}, group number pos^i_{team}, agent property number pos^i_{attr}, and time step pos^i_{time} from which the observation was taken. The structure of these positional vectors depends on the specific environment. For each positional component, a corresponding embedding is learned $emb^i_{type} = L_{type}(pos^i_{type})$ and added to the respective token.

In summary, each observation element o_i is contextualized by a four-component positional vector: the agent index pos^i_{indx}, group identifier pos^i_{team}, attribute type pos^i_{attr}, and timestep pos^i_{time}. While the rules for assigning these positional indices are environment-dependent, the mechanism for using them is universal. Each positional component is mapped to a vector using a learned embedding layer $emb^i_{type} = L_{type}(pos^i_{type})$. These four positional embeddings are then summed with the initial observation token tok_i to produce the final input representation, res_i.

$$res_i = tok_i + emb^i_{indx} + emb^i_{team} + emb^i_{attr} + emb^i_{time}.$$

The result tokens res_i are then processed by a GPT-like model. The example of a positional encoding for the well-known SMACv2 environment is shown in Fig. 3. Appendix A provides recommendations for constructing positional encodings tailored to specific environment.

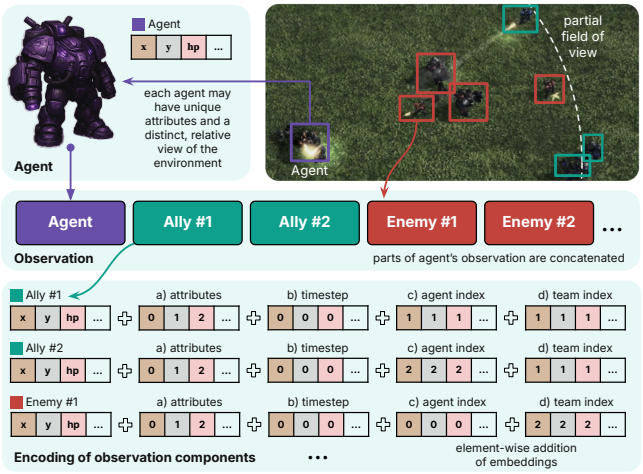

Figure 3: Illustration of the proposed encoding scheme for multi-agent systems, demonstrated using an example from the SMACv2 environment. Each agent receives a structured, vectorized observation containing information about itself, its allies, and nearby enemies. For each observed agent, the input features (such as position, health, etc.) are enriched with additional embeddings: (a) a positional encoding over the feature dimensions, (b) a global timestep encoding, (c) an agent identity index, and (d) a team index distinguishing allies from enemies. These components are combined via element-wise addition, resulting in a contextualized embedding for each observed agent that can be processed by a transformer encoder. The provided encoding scheme is general and transferable across multi-agent systems. By augmenting raw observation features with agent- and team-specific indices, the encoding enables the model to distinguish between different functional groups (e.g., allies vs. enemies) and individual agents, regardless of environment-specific dynamics or layouts. This structure preserves permutation invariance where appropriate, while still allowing the model to learn role-specific interactions. The encoding supports a dynamic number of agents, limited only by the model's maximum context size.

4.3 Model Training

We study a universal model that predicts an agent's action using only its own observation, without access to the state or observations of other agents. Our approach is based on offline RL with a data-driven actor-critic architecture. At each timestep t, the transformer-based observation encoder processes the agent's observation o_t into a hidden state h_t, which is shared by both the critic Q and stochastic policy π output heads. The model is trained by minimizing joint loss $\mathcal{L} = \mathcal{L}_{critic} + \mathcal{L}_{actor}$ using minibatch stochastic gradient descent with the Adam optimizer.

Critic. Classification with categorical cross-entropy loss ($\mathcal{L}_{CE}$) is more effective than regression with mean squared error when training large neural networks, especially transformer architectures. For this reason, we use a discrete critic approach [8, 26]. The Q-value range is divided into a fixed number of uniform bins, and each Q-value is represented as a probability distribution over these bins. If K is the scalar, then K^B represents this scalar as a vector of bins.

The critic loss consists of two parts: the temporal difference (TD) error and the conservative regularization (CR). The TD target is $y_t = r_t + \gamma\ \bar{Q}(o_{t+1}, a \sim \pi(a|o_t))$, where $\bar{Q}$ and $\bar{\pi}$ refer to frozen target networks. The regularization term addresses the issue of overestimating Q-values for observation-action pairs that are rarely or never seen in the dataset [4, 14]. To prevent this, we penalize the Q-values for actions with low probability under the current policy, pushing them toward a minimal attainable value $q_{\min}$. Specifically, we define $\pi'(a \mid o) = \frac{1-\pi(a|o)}{Z}$ as a distribution over actions that have a low probability according to π, where Z is a normalization constant. The critic loss is:

$$\mathcal{L}_{critic} = \alpha_{td}\ \mathbb{E}_{o,a\sim\mathcal{D}}\left[\mathcal{L}_{CE}(Q^B(o,a), y_t^B)\right] + \\ + \alpha_{cr}\ \mathbb{E}_{o\sim\mathcal{D},\ a\sim\pi'(a|o)}\left[\mathcal{L}_{CE}(Q^B(o,a), q_{min}^B)\right]. \quad (1)$$

Policy. The policy loss combines behavior cloning (BC) loss and actor loss (A). We calculate actor loss using the advantage function $A(o_t, a_t)$ because it offers a stable and scale-invariant learning signal. This signal helps the actor focus on actions that perform better than the average, rather than just their absolute value.

$$A(o_t, a_t) = Q(o_t, a_t) - \mathbb{E}_{a\sim\pi(a|o_t)}\left[Q(o_t, a)\right]$$

We include a behavior cloning loss in the policy training because it guides the model to imitate the best behavior observed in the dataset. This loss provides a strong and reliable supervisory signal, especially when the optimal policy is unknown or difficult to estimate. The policy loss is:

$$\mathcal{L}_{actor} = -\alpha_a\ \mathbb{E}_{o_t,a_t\sim\mathcal{D}}\left[A(o_t,a_t)\log\pi(a_t|o_t)\right] \\ + \alpha_{bc}\ \mathbb{E}_{o_t,a_t\sim\mathcal{D},\ a\sim\pi(a|o_t)}\left[\mathcal{L}_{CE}(a, a_t)\right]. \quad (2)$$

Action Space. To handle varying action spaces across environments, a universal output layer (one transformer layer plus a linear layer) is used, combined with environment-specific action masking. This masking ensures that the probability of unavailable actions is always zero, allowing the agent to adapt to different scenarios while maintaining a consistent architecture.

Online fine-tune. During offline training, the agent learns mainly from successful trajectories but tends to underestimate Q-values for unseen observations and actions. This leads to incorrect Q-value estimates during the online phase. To mitigate this, the actor is frozen for the initial n steps of online training while the critic is pretrained using online interactions with the environment. Afterward, online fine-tuning is performed using PPO, which helps stabilize learning and adapt the policy safely to new data.

5 EMPIRICAL EVALUATION

5.1 Experimental Setup

5.1.1 Environments. To evaluate MARL-GPT, we selected three well-known multi-agent environments. SMACv2 [7] is a challenging real-time strategy benchmark that tests coordinated decision-making and teamwork in complex combat. GRF (Google Research Football) [15] provides a dynamic, stochastic setting with varied strategies, assessing policy adaptability and temporal reasoning. The POGEMA benchmark[1] [31] involves multi-agent pathfinding in grid worlds, providing a tough setting for testing scalable generalization to new agent populations and scenarios. We use the lifelong scenario, where a new goal is generated when an agent completes previous one.

5.1.2 Expert Policy Generation. To prepare the expert dataset for SMACv2 and GRF, we used large-scale asynchronous IPPO[2]. In each SMACv2 training environment, IPPO was trained using a total of 1 billion collected observations across scenarios involving *Protoss*, *Terran*, and *Zerg*, with agent populations in both balanced (5vs5, 10vs10) and imbalanced (5vs6) settings. The resulting policy was then used to obtain 400 million agent training samples. We trained expert policies for each GRF scenario using IPPO. For the full 11vs11 match, we trained three distinct expert policies, one for each opponent difficulty level: easy, medium, and hard. Each of these policies was trained for 20 billion environment steps. For the three standard Academy scenarios, *Pass and Shoot with Keeper* ran for 300 million timesteps, *Easy Counter-attack* for 400 million timesteps, and *Corner* for 200 million timesteps. For the POGEMA environment, we used a centralized solver called RHCR [17], in contrast to IPPO, which serves as the expert policy in other settings. RHCR operates by decomposing lifelong MAPF into windowed planning instances, resolving collisions within bounded time horizons while having access to the full state of the environment. The performance of such a centralized policy serves as an upper bound for any learnable policy, which does not have access to the full state.

5.1.3 Trajectory Collection. We then used these pre-trained experts to generate multi-agent trajectories. At every timestep in each scenario, we recorded the observations o_t (as floats), the expert actions a_t, available actions m_t, scalar rewards r_t, and done flags d_t for each agent. The tuple $(o_t, a_t, m_t, r_t, d_t)$ represents a single element in the dataset. For each SMACv2 task, we collected 60 million data elements for tasks with 5 agents and 80 million for those with 10 agents. For GRF this produced approximately 200,000 trajectories from *Pass and Shoot with Keeper*, 25,000 from *Easy Counter-attack*, 75,000 from *Corner*, and 30,000 from the full 11vs11 match. For POGEMA, we collected 1 billion elements in mazes and 120 million on random maps, using 32 agents and 256 steps per episode. We designed a custom dense reward for POGEMA: agents receive 1 when moving toward the goal, and 0 otherwise.

5.1.4 Training. All versions of MARL-GPT were trained using 7M parameters, a history window of 6 (except POGEMA), and the full dataset (except SMACv2 tasks with 10 agents). The main model was trained for 500,000 timesteps, while models used for

[1] https://github.com/Cognitive-AI-Systems/pogema-benchmark
[2] https://github.com/alex-petrenko/sample-factory

Environment	Task	Single-Environment Baselines					Multi-Env	Single-Task
		DT	BC	CQL	BC-LSTM	RATE	MARL-GPT	Expert
SMACv2	protoss 5_vs_5	82 ± 3	61 ± 3	57 ± 3	85 ± 3	79 ± 3	89 ± 3	87 ± 3
	protoss 5_vs_6	30 ± 4	12 ± 4	14 ± 4	30 ± 4	28 ± 4	54 ± 4	49 ± 4
	terran 5_vs_5	84 ± 3	69 ± 3	69 ± 3	88 ± 3	85 ± 3	93 ± 2	91 ± 2
	terran 5_vs_6	48 ± 4	24 ± 4	28 ± 4	51 ± 4	41 ± 4	63 ± 4	61 ± 4
	zerg 5_vs_5	65 ± 3	56 ± 3	50 ± 3	72 ± 3	64 ± 3	74 ± 3	72 ± 3
	zerg 5_vs_6	34 ± 4	24 ± 4	23 ± 4	38 ± 4	33 ± 4	46 ± 4	48 ± 4
GRF	pass and shoot	80 ± 2	44 ± 2	60 ± 2	90 ± 2	78 ± 2	96 ± 2	97 ± 2
	corner	60 ± 5	37 ± 4	30 ± 4	22 ± 4	58 ± 5	43 ± 5	40 ± 4
	counterattack easy	88 ± 3	86 ± 3	87 ± 3	88 ± 3	85 ± 3	89 ± 3	89 ± 3
	11 vs 11 easy	0 ± 0	40 ± 3	38 ± 3	43 ± 4	4 ± 3	98 ± 2	99 ± 1
	11 vs 11 medium	0 ± 0	41 ± 3	35 ± 4	30 ± 3	1 ± 1	98 ± 3	100 ± 0
	11 vs 11 hard	0 ± 0	40 ± 3	34 ± 4	24 ± 5	1 ± 1	68 ± 7	94 ± 1
POGEMA	Random	0.22 ± 0.01	0.24 ± 0.01	0.26 ± 0.01	0.23 ± 0.01	0.21 ± 0.01	1.16 ± 0.04	2.16 ± 0.13
	Mazes	0.12 ± 0.01	0.10 ± 0.01	0.14 ± 0.01	0.11 ± 0.01	0.12 ± 0.01	0.96 ± 0.04	1.55 ± 0.08
	Warehouse	0.13 ± 0.01	0.14 ± 0.01	0.17 ± 0.01	0.12 ± 0.01	0.13 ± 0.01	1.02 ± 0.01	
	Cities-tiles	0.68 ± 0.03	0.46 ± 0.02	0.66 ± 0.03	0.49 ± 0.02	0.65 ± 0.03	2.72 ± 0.12	

Table 1: Win rates % (for SMACv2 and GRF) and average throughput (for POGEMA). Higher is better. The table compares MARL-GPT (Multi-Domain) to expert single-task policies and standard offline RL baselines (Single-Domain) across SMACv2, GRF, and POGEMA. MARL-GPT shows strong generalization, often matching or outperforming the expert and baselines.

testing theories were trained for 10,000. Changes in parameters are explicitly noted. During training, each batch was balanced to provide equal data from all environments, ensuring that the model receives the same proportion of samples per environment. The loss-weighting hyperparameters were fixed and shared across all environments without tuning. The components were chosen to ensure that the actor and critic losses are on a comparable scale, promoting balanced gradient flow. Appendix B includes additional implementation details.

5.1.5 Model evaluation. We assess the model's performance on all tasks within the training distribution for the SMACv2 and GRF domains by running 500 evaluation episodes per task and computing the average win rate. For the pathfinding problem, evaluation is conducted using the POGEMA benchmark on different map sets: random maps and mazes for training tasks, and Warehouse and Cities-tiles for unseen tasks. Performance in POGEMA is measured using the average throughput metric, defined as the ratio of the total number of goals achieved by all agents to the episode length.

5.2 Experimental Results

5.2.1 Multi-task MARL. Table 1 shows the complete experimental results across all three environments. In these experiments, we compare MARL-GPT (trained jointly on all environments) to five standard offline RL baselines (each trained on all tasks within a single environment). The evaluated baselines include two **memory-free** methods: Behavior Cloning (BC) and Conservative Q-Learning (CQL; Kumar et al. [14]); three **memory-based** methods: Decision Transformer (DT; Chen et al. [5]), Recurrent Action Transformer with Memory (RATE; Cherepanov et al. [6]) and a recurrent BC variant with a Long Short-Term Memory [11] backbone (BC-LSTM). Crucially, all offline RL baselines are trained on the same raw observation vectors without the positional encodings: attribute, agent-index, team-index, and timestep that MARL-GPT injects into every token (see 3).

5.2.2 Expert quality. For SMACv2 and GRF, the expert policy is a learned model that is not near-optimal. In contrast, for POGEMA, the expert is a centralized planner (RHCR) that leverages full environment observability and a heavy search to resolve conflicts, providing strong demonstrations that serve as a useful upper bound. We aim to study the components of our model in settings where the expert offers such high-quality supervision.

Table 2 presents results for RHCR, MAPF-GPT (a transformer model trained with behavior cloning on MAPF tasks), MARL-GPT, and MARL-GPT-BC (behavior cloning only). In this setting, behavior cloning variants outperform MARL-GPT. This highlights that MARL-GPT's reward-based loss components may introduce instability when learning from strong expert data, and that designing effective reward functions in multi-agent pathfinding remains challenging due to the need to capture coordination, conflict resolution, and long-term planning. As a result, imitation-based approaches can more effectively leverage high-quality demonstrations like those provided by RHCR.

Scenario	MARL-GPT	MARL-GPT-BC	MAPF-GPT	RHCR
Random	1.16	1.46	1.50	2.16
Mazes	0.96	1.00	1.09	1.55
Warehouse	1.02	1.47	1.27	2.35
Cities-tiles	2.72	2.71	2.99	3.48

Table 2: Average throughput. Comparing MARL-GPT with behavior cloning variants (MARL-GPT-BC, MAPF).

5.2.3 Unseen tasks. For the SMACv2 environment, we evaluated the model on previously unseen tasks with 10vs10 and 7vs7 agents, which are challenging for models trained only on 5 agents. To address this, we collected additional datasets with 10 agents and conducted two experiments: (1) We fine-tuned the pre-trained model on a small (9M) dataset with 10 agents across all races for 2,000 training steps; (2) we trained a new model from scratch using all 5-agent tasks plus an additional terran_10_vs_10 task (for 3,000 and 10,000 training steps). The results (Table 3) show that the model can generalize to new maps with more agents if the training data include tasks with similar agent counts. Moreover, fine-tuning with dataset the pre-trained model requires less data and fewer steps to adapt to a new map than training from scratch. Two experiments show better generalization to unseen tasks. For example, (3) a model trained without terran data still performs well on new terran 5vs5 and 5vs6 scenarios. Additionally, training the model without BC loss across all races (4) can improve performance on tasks with more agents. However, including BC loss helps the model achieve better results on the training tasks but reduces its ability to generalize to new scenarios. All experiments were conducted with history lenghth 4.

Method / Environment	From scratch (2)		Zero-shot			Fine-tune
	3k	10k	MARL-GPT	w/o BC (4)	w/o terran (3)	2k (1)
terran 5_vs_5	87	91	88	80	44*	92
terran 5_vs_6	48	57	52	41	16*	62
terran 10_vs_10	71	83	0*	47*	0*	88
protoss 10_vs_10	62*	61*	0*	8*	0*	81
zerg 10_vs_10	29*	22*	1*	28*	0*	43
terran 7_vs_7	80*	86*	16*	60*	2*	88*

Table 3: Win rates %. Comparing variants of methods on unseen tasks: models trained from scratch and pretrained (zero-shot and fine-tuned). Each experiment used different datasets but always included tasks with zerg and protoss races (5vs5 and 5vs6). Tasks excluded from training and tested zero-shot are highlighted and marked with * next to their success rate. The best zero-shot results are highlighted with a stronger emphasis.

5.2.4 Online fine-tune. Online fine-tune algorithm was tested on unseen tasks, in which MARL-GPT in zero-shot format achieved non-zero results. In SMACv2, we tested online fine-tuning on a new race: the initial weights were trained without the terran dataset (w/o terran in Table 3). Pretraining of the critic used 2M samples. The target task was terran 5_vs_5. At the beginning, the success rate (SR) was 0.4, and it improved to 0.8 by the end (Fig. 4a). For GRF, we tested on several unseen tasks (academy_empty_goal_close, academy_empty_goal and academy_single _goal_versus_lazy) comparing results with an expert trained from scratch. Pretraining of the critic used 500k samples. Fine-tuning the pretrained model on the GRF environment showed faster adaptation to new tasks compared to training a small model from scratch (Fig. 4).

5.2.5 Ablation study. We investigated how different parameters influence the final model's performance (Table 4). All models were trained under identical conditions, except for one varied parameter. MARL-GPT, our final model, uses 7M parameters, a long history window of 6, positional encoding, and the full dataset. Our results show that model size, history length (notably for GRF), training data size, and positional encoding have a substantial impact on performance.

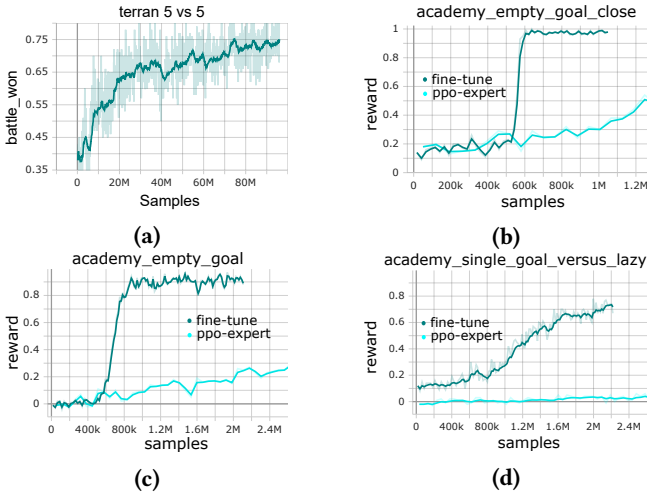

Figure 4: Online fine-tuning results: (a) Battle won on the terran 5vs5 task (SMACv2). (b–d) Comparison of fine-tuned MARL-GPT and an expert model trained from scratch on GRF.

Task	No Pos. Enc.	Model Size		Dataset		History Length		γ
		2M	7M	Half	Small	2	4	0.99
terran 5_vs_6	41	37	53	52	56	49	50	39
zerg 5_vs_6	33	36	40	38	40	37	39	29
corner	29	30	30	32	42	0	32	29
counterattack	88	85	88	87	20	20	0	85
maze	0.56	0.38	0.75	0.70	0.68	0.63	0.66	0.22

Table 4: Ablation study across tasks from SMACv2, GRF, and POGEMA. Reported are win rates (SMACv2 and GRF) and average throughput (POGEMA). We compare MARL-GPT (7M parameters) against a smaller 2M variant, and evaluate the impact of removing positional encoding, reducing dataset size, and varying history length.

5.2.6 Real-World Mini Experiment. We conducted a real-robot MAPF experiment; see Appendix D for details.

6 CONCLUSION

We presented MARL-GPT, a unified transformer-based model for multi-agent reinforcement learning that operates across diverse environments using a single architecture. Trained purely from expert trajectories via imitation learning and RL, MARL-GPT achieves competitive or superior performance compared to specialized baselines in SMACv2, GRF, and POGEMA. Our results demonstrate the viability of a generalist MARL model, suggesting a path forward toward scalable, foundation models for multi-agent decision-making.

REFERENCES

[1] Anton Andreychuk, Konstantin Yakovlev, Aleksandr Panov, and Alexey Skrynnik. 2025. MAPF-GPT: Imitation Learning for Multi-Agent Pathfinding at Scale. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 23126–23134. https://doi.org/10.1609/aaai.v39i22.34477

[2] Raunak P Bhattacharyya, Derek J Phillips, Blake Wulfe, Jeremy Morton, Alex Kuefler, and Mykel J Kochenderfer. 2018. Multi-agent imitation learning for driving simulation. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 1534–1539.

[3] Rishi Bommasani, Drew A Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, et al. 2021. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258* (2021).

[4] Yevgen Chebotar, Quan Vuong, Karol Hausman, Fei Xia, Yao Lu, Alex Irpan, Aviral Kumar, Tianhe Yu, Alexander Herzog, Karl Pertsch, et al. 2023. Q-transformer: Scalable offline reinforcement learning via autoregressive q-functions. In *Conference on Robot Learning*. PMLR, 3909–3928.

[5] Lili Chen, Kevin Lu, Aravind Rajeswaran, Kimin Lee, Aditya Grover, Misha Laskin, Pieter Abbeel, Aravind Srinivas, and Igor Mordatch. 2021. Decision transformer: Reinforcement learning via sequence modeling. *Advances in neural information processing systems* 34 (2021), 15084–15097.

[6] Egor Cherepanov, Alexey Staroverov, Dmitry Yudin, Alexey K Kovalev, and Aleksandr I Panov. 2023. Recurrent action transformer with memory. *arXiv preprint arXiv:2306.09459* (2023).

[7] Benjamin Ellis, Jonathan Cook, Skander Moalla, Mikayel Samvelyan, Mingfei Sun, Anuj Mahajan, Jakob Foerster, and Shimon Whiteson. 2023. SMACv2: An Improved Benchmark for Cooperative Multi-Agent Reinforcement Learning. In *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., 37567–37593. https://proceedings.neurips.cc/paper_files/paper/2023/file/764c18ad230f9e7bf6a77ffc2312c55e-Paper-Datasets_and_Benchmarks.pdf

[8] Jesse Farebrother, Jordi Orbay, Quan Vuong, Adrien Ali Taïga, Yevgen Chebotar, Ted Xiao, Alex Irpan, Sergey Levine, Pablo Samuel Castro, Aleksandra Faust, et al. 2024. Stop regressing: training value functions via classification for scalable deep RL. In *Proceedings of the 41st International Conference on Machine Learning*. 13049–13071.

[9] Roya Firoozi, Johnathan Tucker, Stephen Tian, Anirudha Majumdar, Jiankai Sun, Weiyu Liu, Yuke Zhu, Shuran Song, Ashish Kapoor, Karol Hausman, et al. 2023. Foundation models in robotics: Applications, challenges, and the future. *The International Journal of Robotics Research* (2023).

[10] Adam Fourney, Gagan Bansal, Hussein Mozannar, Cheng Tan, Eduardo Salinas, Friederike Niedtner, Grace Proebsting, Griffin Bassman, Jack Gerrits, Jacob Alber, et al. 2024. Magentic-one: A generalist multi-agent system for solving complex tasks. *arXiv preprint arXiv:2411.04468* (2024).

[11] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long short-term memory. *Neural computation* 9, 8 (1997), 1735–1780.

[12] Chang Huang, Junqiao Zhao, Hongtu Zhou, Hai Zhang, Xiao Zhang, and Chen Ye. 2023. Multi-agent Decision-making at Unsignalized Intersections with Reinforcement Learning from Demonstrations. In *2023 IEEE Intelligent Vehicles Symposium (IV)*. IEEE, 1–6.

[13] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan P Foster, Pannag R Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. 2024. OpenVLA: An Open-Source Vision-Language-Action Model. In *8th Annual Conference on Robot Learning*.

[14] Aviral Kumar, Aurick Zhou, George Tucker, and Sergey Levine. 2020. Conservative q-learning for offline reinforcement learning. *Advances in Neural Information Processing Systems* 33 (2020), 1179–1191.

[15] Karol Kurach, Anton Raichuk, Piotr Stanczyk, Michal Zajac, Olivier Bachem, Lasse Espeholt, Carlos Riquelme, Damien Vincent, Marcin Michalski, Olivier Bousquet, et al. 2020. Google research football: A novel reinforcement learning environment. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 34. 4501–4510.

[16] Hoang M Le, Yisong Yue, Peter Carr, and Patrick Lucey. 2017. Coordinated multi-agent imitation learning. In *International Conference on Machine Learning*. PMLR, 1995–2003.

[17] Jiaoyang Li, Andrew Tinka, Scott Kiesel, Joseph W Durham, TK Satish Kumar, and Sven Koenig. 2021. Lifelong multi-agent path finding in large-scale warehouses. In *Proceedings of the 35th AAAI Conference on Artificial Intelligence (AAAI 2021)*. 11272–11281.

[18] Wei Li, Shiyi Huang, Ziming Qiu, and Aiguo Song. 2024. GAILPG: Multi-Agent Policy Gradient with Generative Adversarial Imitation Learning. *IEEE Transactions on Games* (2024).

[19] Sicong Liu, Yang Shu, Chenjuan Guo, and Bin Yang. 2025. Learning Generalizable Skills from Offline Multi-Task Data for Multi-Agent Cooperation. In *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forum?id=HR1ujVR0ig

[20] Shicheng Liu and Minghui Zhu. 2024. Learning multi-agent behaviors from distributed and streaming demonstrations. *Advances in Neural Information Processing Systems* 36 (2024).

[21] Pablo Alvarez Lopez, Michael Behrisch, Laura Bieker-Walz, Jakob Erdmann, Yun-Pang Flötteröd, Robert Hilbrich, Leonhard Lücken, Johannes Rummel, Peter Wagner, and Evamarie Wießner. 2018. Microscopic traffic simulation using sumo. In *2018 21st international conference on intelligent transportation systems (ITSC)*. IEEE, 2575–2582.

[22] Linghui Meng, Muning Wen, Chenyang Le, Xiyun Li, Dengpeng Xing, Weinan Zhang, Ying Wen, Haifeng Zhang, Jun Wang, Yaodong Yang, et al. 2023. Offline pre-trained multi-agent decision transformer. *Machine Intelligence Research* 20, 2 (2023), 233–248.

[23] Frans A Oliehoek, Christopher Amato, et al. 2016. *A concise introduction to decentralized POMDPs*. Vol. 1. Springer.

[24] Hyunwoo Park, Baekryun Seong, and Sang-Ki Ko. 2025. SPECTra: Scalable Multi-Agent Reinforcement Learning with Permutation-Free Networks. *arXiv preprint arXiv:2503.11726* (2025).

[25] Scott Reed, Konrad Zolna, Emilio Parisotto, Sergio Gómez Colmenarejo, Alexander Novikov, Gabriel Barth-maron, Mai Giménez, Yury Sulsky, Jackie Kay, Jost Tobias Springenberg, Tom Eccles, Jake Bruce, Ali Razavi, Ashley Edwards, Nicolas Heess, Yutian Chen, Raia Hadsell, Oriol Vinyals, Mahyar Bordbar, and Nando de Freitas. 2022. A Generalist Agent. *Transactions on Machine Learning Research* (2022). https://openreview.net/forum?id=1ikK0kHjvj

[26] Anian Ruoss, Grégoire Delétang, Sourabh Medapati, Jordi Grau-Moya, Li Kevin Wenliang, Elliot Catt, John Reid, and Tim Genewein. 2024. Grandmaster-level chess without search. *CoRR* (2024).

[27] Anian Ruoss, Gregoire Deletang, Sourabh Medapati, Jordi Grau-Moya, Li Kevin Wenliang, Elliot Catt, John Reid, Cannada A Lewis, Joel Veness, and Tim Genewein. 2024. Amortized Planning with Large-Scale Transformers: A Case Study on Chess. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.

[28] Mikayel Samvelyan, Tabish Rashid, Christian Schroeder de Witt, Gregory Farquhar, Nantas Nardelli, Tim GJ Rudner, Chia-Man Hung, Philip HS Torr, Jakob Foerster, and Shimon Whiteson. 2019. The StarCraft Multi-Agent Challenge. In *Proceedings of the 18th International Conference on Autonomous Agents and MultiAgent Systems*. 2186–2188.

[29] Andy Shih, Stefano Ermon, and Dorsa Sadigh. 2022. Conditional imitation learning for multi-agent games. In *2022 17th ACM/IEEE International Conference on Human-Robot Interaction (HRI)*. IEEE, 166–175.

[30] David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. 2016. Mastering the game of Go with deep neural networks and tree search. *nature* 529, 7587 (2016), 484–489.

[31] Alexey Skrynnik, Anton Andreychuk, Anatolii Borzilov, Alexander Chernyavskiy, Konstantin Yakovlev, and Aleksandr Panov. 2025. POGEMA: A Benchmark Platform for Cooperative Multi-Agent Pathfinding. In *The Thirteenth International Conference on Learning Representations*.

[32] Jiaming Song, Hongyu Ren, Dorsa Sadigh, and Stefano Ermon. 2018. Multi-agent generative adversarial imitation learning. *Advances in neural information processing systems* 31 (2018).

[33] Jingwu Tang, Gokul Swamy, Fei Fang, and Steven Wu. 2024. Multi-Agent Imitation Learning: Value is Easy, Regret is Hard. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.

[34] Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, et al. 2024. Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213* (2024).

[35] Rishi Veerapaneni, Arthur Jakobsson, Kevin Ren, Samuel Kim, Jiaoyang Li, and Maxim Likhachev. 2024. Work Smarter Not Harder: Simple Imitation Learning with CS-PIBT Outperforms Large Scale Imitation Learning for MAPF. *arXiv preprint arXiv:2409.14491* (2024).

[36] Hongwei Wang, Lantao Yu, Zhangjie Cao, and Stefano Ermon. 2021. Multi-agent imitation learning with copulas. In *Machine Learning and Knowledge Discovery in Databases. Research Track: European Conference, ECML PKDD 2021, Bilbao, Spain, September 13–17, 2021, Proceedings, Part I 21*. Springer, 139–156.

[37] Yutong Wang, Bairan Xiang, Shinan Huang, and Guillaume Sartoretti. 2023. SCRIMP: Scalable communication for reinforcement-and imitation-learning-based multi-agent pathfinding. In *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 9301–9308.

[38] Muning Wen, Jakub Kuba, Runji Lin, Weinan Zhang, Ying Wen, Jun Wang, and Yaodong Yang. 2022. Multi-agent reinforcement learning is a sequence modeling problem. *Advances in Neural Information Processing Systems* 35 (2022), 16509–16521.

[39] Liming Xu, Sara Almahri, Stephen Mak, and Alexandra Brintrup. 2024. Multi-agent systems and foundation models enable autonomous supply chains: Opportunities and challenges. *IFAC-PapersOnLine* 58, 19 (2024), 795–800.

[40] Fan Yang, Alina Vereshchaka, Changyou Chen, and Wen Dong. 2020. Bayesian multi-type mean field multi-agent imitation learning. *Advances in Neural Information Processing Systems* 33 (2020), 2469–2478.

[41] Sherry Yang, Ofir Nachum, Yilun Du, Jason Wei, Pieter Abbeel, and Dale Schuurmans. 2023. Foundation models for decision making: Problems, methods, and opportunities. *arXiv preprint arXiv:2303.04129* (2023).

[42] Yiming Zhang, Kun Yang, Cong Shen, and Dongning Guo. 2025. Multi-Agent Decision Transformer for Power Control in Wireless Networks. In *2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. 1–5.

7 AAAI REVIEWS

7.1 Phase 2 AC Recommendation by Area Chairs

Metareview: The paper does a great job in several aspects, but as raised by the reviewer(s) and SPC, there are a few issues including limited novelty, under-detailed technical aspects, and limited analysis (baselines), that hold it back from realizing its potential. We hope that the reviews provide guidance such that when addressed the paper might one day stand as a firm state-of-the-art approach.

Acceptance Recommendation: This paper is slightly below papers presented at a top tier venue like AAAI. (Weak reject recommendation.)

Confidence: 4: The AC is confident but not absolutely certain.

7.2 Phase 2 SPC Recommendation by Senior Program Committee Zxqu

Metareview: Ambitious and timely attempt to train a single transformer policy across SMACv2/GRF/POGEMA with a unified observation encoding and large offline expert data; results are competitive and the engineering pipeline is clear. However, novelty is largely a recombination of known pieces, "foundation model" claims outpace evidence, and practicality hinges on massive expert datasets plus non-trivial manual encoding. Key technical specs (loss definitions, discrete-critic details, action-space handling) are underspecified; baseline breadth/fairness is limited (missing recent offline MARL and parity on tokenization); zero-shot generalization is weak; and compute/overhead reporting is sparse.

Acceptance Recommendation: This paper is slightly below papers presented at a top tier venue like AAAI. (Weak reject recommendation.)

Confidence: 4: The SPC is confident but not absolutely certain.

7.3 R1. Official Review of Submission 20365 by Program Committee BCUx

Review: This paper develops a transformer-based foundation model for **cooperative** multi-agent reinforcement learning (MARL) tasks, including SMACv2, GRF, and POGEMA. Specifically, the training process consists of three sub-processes: (1) Training RL-based expert policies, or rule-based expert policies for all single-task scenarios; (2) Collecting a large number of offline expert demonstrations across all scenarios; (3) Training the transformer-based foundation model (universal policy) using RL loss and behavior cloning loss. Furthermore, extensive experiments demonstrate that the proposed foundation model exhibits superior performance under both multi-task and zero-shot settings.

There are several remaining issues that should be addressed:

- In the second paragraph within the Sec. Introduction, SMAC, GRF are both **cooperative** MARL tasks (while the authors state that they are adversarial games), where the enemies are controlled by in-build AI. This should be highlighted to place the core contribution of this paper to cooperative MARL.
- In the Sec. Background, the authors state that agents select actions based on its own local action-observation history. However, the partial observability does not hold within the definition of Markov Game. The author should give a comprehensive introduction to POSG / POMG or Dec-POMDP (often for cooperative MARL tasks), rather than Markov Game.
- The Sec. Observation Encoding contains excessive verbal descriptions and exhibits misuse of symbols. Specifically, the observation from any environment can be represented as a vector $o = \{o_i\}_{i=0}^{n}$. This refers to observations of all agents $\{1, 2, ..., n\}$ or local observation of any single agent? This induces ambiguous notations. In addition, the content of the penultimate and antepenultimate paragraphs is redundant, specifically regarding the statement that "each observation element can be represented by a tuple of four components." These issues must be eliminated.
- The primary contributions of this work are threefold: (1) the construction of a large-scale dataset comprising offline expert demonstrations across multiple tasks; (2) the establishment of a shared observation representation space; and (3) the training of a transformer-based universal policy. The design of (2) should account for different task configurations; specifically, it must address scenarios where no distinction is made between allies and enemies within a task. This work is intended to serve as a guiding and pioneering framework for the development of subsequent foundation models in MARL. In its current form, the exposition of (2) relies excessively on textual descriptions, lacking sufficient formal notation or supporting diagrams. The reviewer suggests that this section requires a revised layout and substantive content modifications. Alternatively, the authors could explicitly state that the proposed foundation model is designed specifically for cooperative tasks involving clear ally-enemy distinctions.
- The reviewer recommends that the manuscript be further revised and substantiated in terms of both content and layout: (1) augmenting the exposition of related work in cooperative MARL to better highlight the paper's contribution to the MARL domain, rather than focusing predominantly on Multi-Agent Imitation Learning (MAIL); (2) revising the methodology section by incorporating a schematic diagram to illustrate the proposed approach; and (3) assigning a specific **name** to the proposed foundation model, thereby formalizing its methodological identity. It is advisable to consult exemplary publications from the computer vision field, such as the "Depth Anything [1]" article, for guidance on effective paper structuring and presentation.

In terms of its novelty on introducing foundation model for cooperative MARL, the reviewer acknowledges that the current manuscript meets the threshold for marginally above acceptance. However, its potential for a higher rating is constrained by deficiencies in both content depth and structural organization.

References: [1] Yang L, Kang B, Huang Z, et al. Depth anything: Unleashing the power of large-scale unlabeled data[C]//Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. 2024: 10371-10381.

Rating: 6: Marginally above acceptance threshold

Confidence: 5: The reviewer is absolutely certain that the evaluation is correct and very familiar with the relevant literature

7.4 R2. Official Review of Submission 20365 by Program Committee qR97

Review: This paper proposes a general large model for diverse MARL tasks, where the trained model can achieve desired performance across different tasks without requiring task-specific finetuning. The training methodology is straightforward, employing only behavioral learning to train the large model. The paper is well-written with clear motivation and comprehensive methodological details. However, I have several concerns that could help improve the paper's quality.

Ablation Study for Observation Encoding: Since the authors identify Observation Encoding as a crucial component of their approach, they should verify its effectiveness through ablation studies comparing performance with and without this module.

Insufficient Baseline Comparisons: The baseline methods should be expanded to include state-of-the-art algorithms from recent years, such as COMADICE [1] and INS [2], to provide a more comprehensive evaluation of the proposed method's performance.

Performance Inconsistency Across Environments: The performance disparity requires explanation—why does the approach perform significantly worse than expert policies in GRF and POGEMA environments while achieving good performance in SMACv2? This inconsistency raises questions about the method's generalizability claims.

Figure Quality Issues: Figures 2 and 3 are excessively large. I recommend reducing their size to ensure that the font size within the figures does not exceed that of the main text, improving readability and professional presentation.

Dataset Quality Dependency: The approach appears to rely heavily on high-quality expert datasets. It would be valuable to investigate and discuss how performance degrades when such high-quality training datasets are unavailable, as this limitation could significantly impact the method's practical applicability.

[1] Nguyen T H, Mai T A. Comadice: Offline cooperative multi-agent reinforcement learning with stationary distribution shift regularization[C]//The Thirteenth International Conference on Learning Representations. 2025.

[2] Fu Y, Zhu Y, Zhao J, et al. INS: Interaction-aware Synthesis to Enhance Offline Multi-agent Reinforcement Learning[C]//The Thirteenth International Conference on Learning Representations. 2025.

Rating: 6: Marginally above acceptance threshold

Confidence: 4: The reviewer is confident but not absolutely certain that the evaluation is correct

7.5 R3. Official Review of Submission 20365 by Program Committee ctQd

Review:

Strengths:

- The paper proposes a novel approach that uses transformer-based architectures for solving many MARL tasks using the same model. I believe that the approach would be very interesting for the MARL community.
- The proposed framework is decentralized, and thus is not built upon the widely used (though not so realistic) CTDE schema.

Weaknesses:

- The intoduction needs better motivation for why the proposed method is novel in the context of related work. Maybe the authors should consider merging some parts of the related work with the introduction. I believe that this work can be improved a lot by improving the overall motivation of the paper.
- The observation encoding is somewhat heuristic and maybe also requires significant effort (and new ideas) to code for new environments. Have the authors tried more abstract representations, e.g., via VAEs., instead of the four proposed components?
- One concern is that MARL-GPT does not significantly improve over the expert policy (which is a well-trained IPPO in most cases). So, what is the main point (apart from multi-tasking) to use all this technology instead of standard MARL baselines?
- One other concern is whether the model can be trained without imitation learning and only via the RL objective. What is the main asset of the model: imitation learning (and thus IPPO) or the transformer (GPT) architecture?

Further Suggestions:

- I would suggest the authors cite works highlighting the importance of the selected testbeds and widely used techniques, such as parameter sharing, (e.g., see [1]) and multi-agent actor-critic algorithms (e.g. see [2]).
- It would be better to also cite behavior cloning in the Background section.
- It would be also beneficial to see how fast the model converges to its "best" policy. I would recommend the authors add such plots and further compare with baselines.

References:

[1] Papoudakis, Georgios, et al. "Benchmarking Multi-Agent Deep Reinforcement Learning Algorithms in Cooperative Tasks." Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track.

[2] Papadopoulos, George, et al. "An Extended Benchmarking of Multi-Agent Reinforcement Learning Algorithms in Complex Fully Cooperative Tasks." Proceedings of the 24th International Conference on Autonomous Agents and Multiagent Systems. 2025.

Rating: 6: Marginally above acceptance threshold

Confidence: 3: The reviewer is fairly confident that the evaluation is correct

7.6 R4. Official Review of Submission 20365 by Program Committee GGtX

Review: This paper introduces MARL-GPT, a transformer-based foundation model for multi-agent reinforcement learning (MARL). The model is trained on large, pre-collected expert datasets from diverse environments (SMACv2, GRF, POGEMA) using a novel, unified observation encoding scheme. Experiments show the single model achieves competitive performance against single-domain baselines.

The paper's primary strength is its novelty in pursuing a single, generalist MARL foundation model, a significant step beyond

domain-specific solutions. This is supported by the empirical evaluation on challenging benchmarks. The proposed observation encoding scheme is an effective method for unifying heterogeneous inputs, and the commitment to open-sourcing all assets is a major contribution. However, the work is more a novel application of existing techniques rather than a new invention. The "GPT-based" descriptor may be somewhat misleading, as it is a non-generative transformer encoder. The model's success is fundamentally limited by the availability of massive, high-quality expert datasets, and the observation encoding requires non-trivial, manual feature engineering per domain, which hinders scalability.

Questions:

(1) How much manual effort was required to design the positional encoding scheme for each of the three environments?
(2) The results in Table 1 are impressive, particularly where MARL-GPT surpasses the expert policy. Could you provide more insight into how an offline-trained agent can learn a superior policy from a fixed dataset of demonstrations?
(3) What were the computational resources (e.g., total GPU hours) required to train the final MARL-GPT model on the full 1.5B+ transition dataset?
(4) The fine-tuning results in Table 3 are promising. When fine-tuning on the 10-agent SMACv2 data, did you observe any performance degradation on the original tasks from GRF or POGEMA?
(5) How does the model architecture handle scenarios where the number of agents changes during an episode?
(6) Could you elaborate on how the model's output head handles the different action spaces of each environment? Is it a single, large linear layer with a shared vocabulary and action masking, or a different mechanism?
(7) The authors briefly mention a "Real-World Mini Experiment." Could you provide a summary of this experiment and its outcome?
(8) How were the loss-weighting hyperparameters (α_a, α_{bc}, etc.) selected? Was a single set of values used for the joint training across all environments? How sensitive is the final model's performance to these weights?

Minors:

- Table 1: Color the other baselines too that lie within the confidence interval.
- Table 3 caption: "* tasks was not in the train dataset." -> "* tasks were not in the training dataset." Actually, it is unclear how a task can be "not in the train dataset" if the model is being trained from scratch on a dataset defined for that experiment. Please clarify the source tasks.

Rating: 5: Marginally below acceptance threshold

Confidence: 4: The reviewer is confident but not absolutely certain that the evaluation is correct

7.7 R5. Official Review of Submission 20365 by Program Committee gHSG

Review: **Summary**: This paper proposes MARL-GPT, a transformer-based foundation model for Multi-Agent Reinforcement Learning (MARL), aiming to achieve general decision-making capabilities across multiple significantly diverse MARL environments using a unified architecture. The method combines large-scale expert trajectory data, a flexible observation encoding mechanism, and a training strategy based on offline reinforcement learning and imitation learning, achieving competitive or superior performance compared to specialized baselines in three environments: SMACv2, GRF, and POGEMA. This work demonstrates considerable innovation and practical potential in the MARL field, particularly in advancing the development of generalist agent models.

Strengths:

- The writing is clear, and the methodological pipeline is well-structured and easy to follow. The process of acquiring expert data, designing a generalizable observation encoding, and training with a combined imitation and offline RL objective is logically presented.
- The core objective of developing a single transformer-based model for vastly different MARL benchmarks (SMACv2, GRF, POGEMA) is highly relevant and ambitious. The empirical validation is extensive, demonstrating competitive performance across these domains and including important analyses on unseen tasks and ablations.

Weaknesses:

- The performance appears heavily dependent on the quality and scale of the expert data, which requires enormous computational resources to generate. The practical feasibility of applying this method to new domains, where such expert policies are not readily available, is not sufficiently discussed. The wall-clock time for inference and the potential for online fine-tuning are also left unaddressed.
- While the application of a Transformer architecture to multi-task MARL is novel, the core architectural components (behavior cloning, discrete Q-value bins, conservative regularization) are combinations of existing techniques from offline RL and sequence modeling. The paper could more sharply highlight what is uniquely innovative in its architectural design compared to prior work like DT or Q-Transformer.

Questions:

(1) How does MARL-GPT perform on completely unseen environments (outside the training distribution)? Are there plans to evaluate its zero-shot or few-shot capabilities?
(2) Does the model support online fine-tuning? That is, can it be further optimized through interaction with the environment after deployment?
(3) How does the observation encoding handle dynamically changing numbers of agents (e.g., agent death or spawning)? Does the current encoding scheme have limitations here?
(4) In the POGEMA task, why does the pure Behavior Cloning (BC) variant outperform the RL-augmented MARL-GPT? Does this suggest that the RL components introduce noise or instability in tasks with strong expert demonstrations?
(5) Are there plans to extend MARL-GPT to scenarios involving explicit communication or partially centralized decision-making?

Rating: 5: Marginally below acceptance threshold

Confidence: 3: The reviewer is fairly confident that the evaluation is correct

8 AAMAS SUBMISSION UPDATES

We sincerely thank the reviewers for their valuable feedback and appreciate their recognition of our model, demonstrating generalist capabilities across diverse environments. We address the key concerns as follows.

Because the reviewers noted that the results were weak owing to the model's inability to adapt to new tasks, an essential feature of a fundamental model, we performed additional experiments to investigate and clarify this aspect.

- **General answer:** Zero-shot capabilities were demonstrated in a single environment (POGEMA: warehouse, Cities-tiles; SMACv2: terran7vs7, protoss10vs10, etc.), but new environments require additional fine-tuning. The Actor-Critic architecture will enable online fine-tuning on unseen tasks in future work.
- **Generalization to unseen tasks (R2,R5):** We expanded Section 5.2.3 with an extended discussion of generalization and added new experiments on unseen tasks in the zero-shot setting. We also revised Table 3 to more clearly reflect the experimental results and to explicitly highlight the zero-shot performance.
- **Online fine-tuning (R4,R5):** We added Section 5.2.4, which presents the results of online fine-tuning MARL-GPT on previously unseen tasks. The results in Figure 4 show that on SMACv2 the success rate doubles (from 0.4 to 0.8), while on GRF the fine-tuning provides a substantial improvement, drastically outperforming training the model from scratch on the same tasks. Thus, MARL-GPT demonstrates the properties of a foundation model, allowing it to efficiently adapt to new, previously unseen tasks thanks to large-scale pretraining.

Therefore, our experiments with fine-tuning and zero-shot methods showed that the model can be effectively adapted to new tasks by additional training, either online or using a small dataset. This method outperforms training the model from scratch.

Dec-POMDP (R1): We agree with Reviewer 1 that framing our setting as a Dec-POMDP is indeed relevant, so we have revised the problem statement accordingly. The Dec-POMDP framework emphasizes that the model relies solely on the local observation of one single agent.

Manual positional encoding (R3,R4,R5): Positional encoding requires a manual setup once, which can be quickly done using information about the agent's observation structure from the environment (see Appendix A for instructions). For the entire SMACv2 environment, this setup is implemented as a concise single-page function. Additionally, this process can be assisted by a large language model (like ChatGPT), which can help generate the encoding. The advantage of this positional encoding is that it only needs to be configured once for any environment, without requiring changes to the model architecture. Traditional positional encoding methods tie features to fixed, task-specific positions in observations, requiring additional adjustments when the number of agents or features change. Our encoding removes this limitation, allowing the model to adapt seamlessly to environments with varying agent counts and properties, either within a single episode or across different tasks.

Performance with a strong expert (R2,R4,R5): Using an Actor-Critic method with an imperfect expert highlights where the expert's behavior can be improved. This capability, with cross-task knowledge transfer, enables learning a policy that surpasses the expert (e.g., SMACv2). The quality of expert data matters: PPO-based experts are noisy and can make errors. Strong experts provide consistently good actions, making it hard to extract a useful reward signal (there is no suboptimal behaviour to improve) since no exploration behaviour is presented in the dataset. Thus, RL adds some instability in policy with strong expert. The reviewer's suggestion to test our method with experts of varying quality is highly valuable. Conversely, strictly imitating the expert can limit the model's ability to generalize to new tasks and environments. This effect is illustrated in an additional experiment in Table 3, where reducing the influence of the BC loss causes a slight decrease in performance on training tasks but leads to improved results on tasks involving more agents in the SMACv2 environment.

Training details (R4): The loss-weighting hyperparameters were fixed and shared across all environments without tuning. The components were chosen to ensure that the actor and critic losses are on a comparable scale, promoting balanced gradient flow. Performance degrades if one loss dominates. We fine-tuned on 10-agent SMACv2 data mixed with batches from original tasks to prevent catastrophic forgetting, so no performance drop occurred. All parts of this answer have been included in the main text of the article and in the appendix..

Action spaces (R4): We added explanations in Section 4.3 about how MARL-GPT operates with multiple action spaces simultaneously. We use a single universal output transformer layer+linear layer combined with environment-specific action masking to handle varying action spaces.

Here is the list of key differences and enhancements in the AAMAS submission that address the AAAI reviews:

- Clarified several missing details, including the description of the action space.
- Updated the problem statement to a Decentralized Partially Observable Markov Decision Process (Dec-POMDP).
- Added an additional experiment demonstrating the proposed approach's capabilities in online fine-tuning.
- Added an additional experiment demonstrating the zero-shot capabilities of our model (Table 3).
- Changed a size of figures.

9 AAMAS RELEVANCE STATEMENT

This submission is highly relevant to the AAMAS community because it advances the study of multi-agent reinforcement learning, a core topic within the **Learning and Adaptation** track. The proposed MARL-GPT framework introduces a unified, transformer-based architecture capable of learning effective behaviors across heterogeneous multi-agent environments without task-specific engineering. By demonstrating that a single model can generalize across SMACv2, GRF, and POGEMA using large-scale offline RL and imitation learning, the work directly contributes to key LEARN

themes such as **multiagent learning**, **supervised learning**, and **reinforcement learning**. The results move the field toward scalable, general-purpose MARL foundation models, an emerging direction of high interest to the AAMAS research community.